14b — Suchalgorithmen

Linear und binär – warum sich Sortieren lohnt

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

Agenda

1. Wozu Suchalgorithmen?
2. Lineare Suche – Element für Element
3. Binäre Suche – Halbieren statt Durchlaufen
4. Unter der Haube: Invariante & $\lceil \log_2 n \rceil$
5. Vorbedingung Sortiertheit – Rückbezug auf 14a
6. Linear vs. binär – der Vergleich
7. **Klassiker: Der Computer errät deine Zahl** – live programmiert

Lernziel: beide Suchverfahren selbst implementieren – und **begründen**, warum Halbieren aus 1 000 000 Kandidaten in 20 Schritten einen macht.

Heute: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 14b).

Wozu Suchalgorithmen?

Veggie Soles hat 1 000 000 Bestell-IDs – wo steckt Bestellung 4711?

Ohne Plan: alles durchgehen

```
for i in range(len(ids)):      # schlimmstenfalls
    if ids[i] == 4711:         # 1 000 000 Vergleiche
        ...
```

Mit Plan: geschickt halbieren

Sortierte Liste + binäre Suche $\longrightarrow$ höchstens **20** Vergleiche.

Heute: zwei Algorithmen, **gleiches Ergebnis** – aber $O(n)$ gegen $O(\log n)$. Der Unterschied entscheidet, ob eine Suche spürbar dauert oder nicht.

Lineare Suche – Element für Element

```python
def lineare_suche(liste, ziel):
    for index in range(len(liste)):
        if liste[index] == ziel:
            return index       # erster Treffer
    return -1                  # nicht gefunden
```

- Von **vorn nach hinten**, jedes Element vergleichen
- Funktioniert auf **jeder** Liste – sortiert oder nicht
- Konvention: `-1` signalisiert “nicht gefunden” (kein gültiger Index)

`ziel in liste` macht intern **genau diese Schleife** – nur ohne den Index zurückzugeben.

Unter der Haube: lineare Suche formal

```
ids = [101, 207, 314, 422, 555]
lineare_suche(ids, 555)    # 5 Vergleiche
lineare_suche(ids, 999)    # 5 Vergleiche, dann -1
```

Fall	Vergleiche	Big-O
best: Ziel ganz vorn	1	$O(1)$
typisch: Ziel in der Mitte	ca. $n/2$	$O(n)$
worst: Ziel hinten oder fehlt	n	$O(n)$

Trotzdem oft die **richtige** Wahl: bei **unsortierten** Daten und **Einmal-Suchen** wäre erst zu sortieren teurer als die eine Suche selbst.

Lineare Suche – Demo & Übung

▶ **Live-Demo** – `01_lineare_suche.py` (Print-Trace pro Vergleich), `02_lineare_suche_cursor.py` (Cursor `^pos` wandert durch die Liste)

✎ **Sofort ausprobieren** – Notebook 14b, Kap. 4.1: `lineare_suche` nachbauen, mit Ziel vorn / hinten / fehlend testen

Binäre Suche – Halbieren statt Durchlaufen

Wie im Telefonbuch: in der **Mitte** aufschlagen, dann nur noch in der richtigen Hälfte weitersuchen.

```
Suche 7 in [1, 2, 3, 4, 5, 6, 7, 8]
Mitte 4:  7 > 4  -> rechte Haelfte
Suche in [5, 6, 7, 8]
Mitte 6:  7 > 6  -> rechte Haelfte
Suche in [7, 8] -> Mitte 7: Treffer!
```

- **Ein** Vergleich wirft die **halbe** Liste weg
- Voraussetzung: die Liste ist **sortiert** (gleich mehr dazu)

Binäre Suche – der Code

```
def binaere_suche(liste, ziel):
    links, rechts = 0, len(liste) - 1
    while links <= rechts:
        mitte = (links + rechts) // 2
        if liste[mitte] == ziel:
            return mitte
        elif liste[mitte] < ziel:
            links = mitte + 1
        else:
            rechts = mitte - 1
    return -1
```

- `links`/`rechts` begrenzen den **Suchraum**; `//` hält `mitte` ganzzahlig
- Ende: Treffer (`return mitte`) oder leeres Intervall (`links > rechts`)

Unter der Haube: die Invariante der binären Suche

Invariante: Ist das Ziel in der Liste, dann liegt es **in jedem Moment** im Intervall $[$`links`, `rechts`$]$.

Suche 16 in `[2, 5, 8, 12, 16, 23, 38, 56]`:

Schritt	links	mitte	rechts	`liste[mitte]`	Folge
1	0	3	7	12 < 16	`links = 4`
2	4	5	7	23 > 16	`rechts = 4`
3	4	4	4	16 = Ziel	**Treffer!**

- Jede Halbierung **erhält** die Invariante: verworfen wird nur, wo das Ziel sicher **nicht** ist – das geht **nur, weil sortiert ist**
- Wird `links > rechts`: Intervall leer $\rightarrow$ Ziel war nie da $\rightarrow$ `-1`

Unter der Haube: warum $\lceil \log_2 n \rceil$ Schritte?

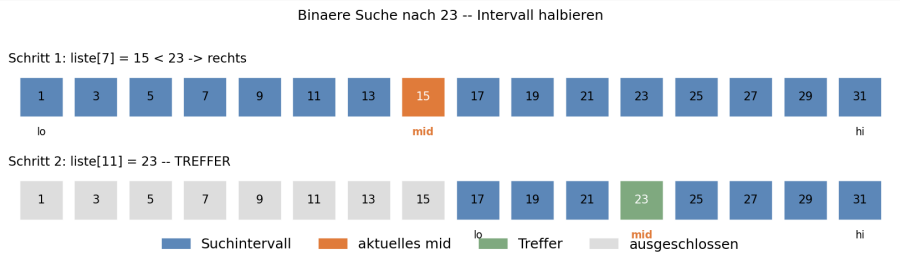
Jeder Schritt halbiert den Suchraum, bis ein Kandidat übrig ist:

$$n \;\to\; \frac{n}{2} \;\to\; \frac{n}{4} \;\to\; \cdots \;\to\; 1$$

n	max. Schritte $\lceil \log_2 n \rceil$
100	7
1 000	10
1 000 000	20
1 000 000 000	30

Verdopplung der Datenmenge kostet nur **einen** Schritt mehr – die Superkraft von $O(\log n)$.

Binäre Suche – das Intervall *sehen*



Suche nach 23: Schritt 1 verwirft die halbe Liste, Schritt 2 trifft – erlaubt wären bei 16 Elementen bis zu $\lceil \log_2 16 \rceil = 4$ Schritte.

▶ **Live-Demo** – `03_binaere_suche_intervall.py`: drei Zeiger `^lo`, `^mid`, `^hi` wandern live im Terminal

Unter der Haube: Vorbedingung Sortiertheit

Binäre Suche auf **unsortierter** Liste – Suche 7 in `[42, 7, 93, 15, 61]`:

Schritt	links	mitte	rechts	Vergleich	Folge
1	0	2	4	93 > 7	`rechts = 1`
2	0	0	1	42 > 7	`rechts = -1`
–	0	–	-1	Intervall leer	Ergebnis -1

- Ergebnis “nicht gefunden” – obwohl die 7 an Index 1 steht!
- **Kein Absturz, keine Warnung**: stillschweigend falsch – der gefährlichste Fehlertyp

Deshalb 14a vor 14b: **erst sortieren** (einmalig, $O(n \log n)$), **dann binär suchen** ($O(\log n)$ pro Suche) – das lohnt, sobald mehrfach gesucht wird.

Linear vs. binär – der Vergleich

	lineare Suche	binäre Suche
Vergleiche (worst)	n	$\lceil \log_2 n \rceil$
bei $n = 1\,000\,000$	1 000 000	20
Voraussetzung	keine	Liste **sortiert**
Code	5 Zeilen, 1 Cursor	11 Zeilen, 3 Zeiger

Bei $n = 1\,000\,000$ ist binär im worst case **50 000-mal** schneller.

▶ **Live-Demo** – `04_linear_vs_binaer.py`: beide Algorithmen auf 1 Million Bestell-IDs, Schritte + Laufzeit gemessen

Wann nehme ich welche?

Situation	Empfehlung
unsortiert, einmal suchen	linear – Sortieren lohnt nicht
oft suchen, selten ändern	einmal sortieren + binär
kleine Liste ($n < 100$)	linear – Unterschied unspürbar
Nachschlagen per Schlüssel	`dict`/`set`: `in` im Schnitt $O(1)$

✎ **Sofort ausprobieren** – Notebook 14b, Kap. 4.2: `binaere_suche` implementieren und die Laufzeit gegen `lineare_suche` auf 1 Mio. Werten messen

Klassiker: Der Computer errät deine Zahl – die Aufgabe

Das Finale von Foliensatz 10 mit **vertauschten Rollen**: damals haben **Sie** geraten – heute rät **der Computer**.

- **Sie** denken sich eine Zahl zwischen 1 und 100
- Das Programm rät per **Intervallhalbierung** und stellt Fragen
- Sie antworten: g (zu groß), k (zu klein) oder = (richtig)
- Am Ende: Fragenzähler – nie mehr als $\lceil \log_2 100 \rceil = 7$ Fragen!

So soll es laufen

```
Denk dir eine Zahl von 1 bis 100.
Ist es 50? (g/k/=): k
Ist es 75? (g/k/=): g
Ist es 62? (g/k/=): =
In 3 Fragen erraten!
```

Klassiker: Der Computer errät deine Zahl – Live-Coding

Das ist **binäre Suche** auf der Liste $1, 2, \ldots, 100$ – Ihre Antworten ersetzen `liste[mitte]`.

▶ **Live-Coding ab leerem Editor** – Referenz: `90_klassiker_computer_raet.py`
Teilziele:

1. Intervallgrenzen: `links = 1`, `rechts = 100`
2. Frage stellen: `mitte = (links + rechts) // 2`
3. Antwort einlesen: g $\rightarrow$ `rechts = mitte - 1`, k $\rightarrow$ `links = mitte + 1`
4. = $\rightarrow$ `break` + Fragenzähler ausgeben
5. Invariante checken: die gedachte Zahl liegt immer in `[links, rechts]`

✎ **Zum Selberlösen** – Handout `Klassiker/14b_Klassiker_Computer_Raet.pdf`, inkl. Schummel-Bonus

Cheat Card

Aufgabe	Syntax
Lineare Suche	`for i in range(len(liste)):`
Binäre Suche: Start	`links, rechts = 0, len(liste) - 1`
solange Intervall da	`while links <= rechts:`
Mitte	`mitte = (links + rechts) // 2`
Halbieren	`links = mitte + 1` / `rechts = mitte - 1`
`x in liste`	lineare Suche, $O(n)$
`x in dict` / `set`	Hash-Zugriff, im Schnitt $O(1)$
Klassiker heute	Intervall halbieren, max. 7 Fragen

Faustregel: einmal suchen $\rightarrow$ linear; oft suchen $\rightarrow$ sortieren + binär – oder gleich `dict`/`set`.

Ausblick: Notebook 15 – strukturierte Programme

Unser Klassiker funktioniert – aber alles liegt lose in **einem** Skript:

```
links = 1
rechts = 100
# ... Spiel-Logik direkt im Skript
```

- Ratespiel, Suche, Sortierung: lauter wiederverwendbare Bausteine – aber nur, wenn sie sauber **verpackt** sind
- Notebook 15 (Ausblick): Programme strukturieren mit `main()` und Funktionen als Bausteine – Modul vs. Skript
- Danach (ab NB 16): Datenanalyse mit Pandas – dort stecken Sortieren und Suchen in jedem Arbeitsschritt

Heute geübt

- ✓ Lineare Suche: $O(n)$, funktioniert auf jeder Liste
- ✓ Binäre Suche: Intervall `[links, rechts]` halbieren, $O(\log n)$
- ✓ Die Invariante – **warum** binäre Suche korrekt ist
- ✓ $\lceil \log_2 n \rceil$: 1 Million Elemente $\rightarrow$ 20 Schritte
- ✓ Vorbedingung Sortiertheit – sonst stillschweigend falsche Ergebnisse
- ✓ **Klassiker: Der Computer errät deine Zahl** live gesehen

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker eigenständig lösen (Handout, inkl. Schummel-Bonus)
- Notebook 14b: beide Suchverfahren implementieren + Laufzeit messen
- Kopfrechnen-Check: wie viele Fragen für 1 bis 1000? (Es sind 10.)